Algoritmi e strutture dati

Francesco Lanzarini

1 Notazione asintotica

Non ci interessiamo direttamente al tempo esatto di calcolo dell'algoritmo ma la crescita del tempo richiesto rispetto alla crescita dell'input. Questa notazione permette di confrontare rapidamente tra loro algoritmi differenti che risolvono lo stesso problema, indipendentemente dall'hardaware.

1.1 Definizioni e notazioni matematiche

Funzione di costo Associa ad ogni input n la quantità di calcolo necessaria a terminare il calcolo, tutti i numeri sono naturali quindi un numero di bit non negativo e non frazionario ma il codomionio delle funzioni è $\mathbb{R}^+$. Una volta trovata la funzione di costo ne si sstudia l'andamento asintotico per capirne il **tasso di crescita**.

O-grande Data una funzione $g(n)$ l'insieme delle funzioni $O-grande$ è composto da tutte quelle funzioni con un ordine di crescita inferiorie di $g(n)$; $g(n)$ è dunque il **limite asintotico superiore**.

Ω-grande Data sempre $g(n)$ con $\Omega-grande$ si indica l'insieme delle funzioni con ordine di crescita maggiore o uguale di $g(x)$; $g(x)$ e' dunque il **limite di crescita inferiore**.

Θ Data sempre $g(n)$ si indica con Θ l'insieme di funzioni che hanno lo stesso ordine di crescita di $g(n)$, in altre parole $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$

o-piccolo La stessa definizione data in analisi ma più specifica essendo in questo caso applicata alle funzioni di costo.

ω-piccolo Ovvero l'insieme di funzioni che dominano asintoticamente $g(n)$

Complessità computazionale di un algoritmo Un algoritmo A ha complessità computazionale $O(f(n))$ rispetto ad una certa risorsa di calcolo se la quantità di risorse necessaria per eseguire A su un qualsiasi input di dimensione n è $O(f(n))$.

Complessità computazionale di un problema Un problema P ha complessità computazionale $O(f(n))$ rispetto ad una certa risorsa di calcolo se il migliore algoritmo che risolve P ha costo computazionale $O(f(n))$ rispetto a tale risorsa di calcolo. E' molto più complicata da calcolare rispetto a quella di un algoritmo.

1.2 Scopo di queste definizioni

L'ordine di crescita asintotico può essere confrontato attraverso le notazioni di limite, se il limite all'inifnito tra $f(n)$ e $g(n)$ esiste si presentano 3 casistiche:

- $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ allora $f(n) = o(g(n)) \Rightarrow f(n) = O(g(n))$
- $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$ allora $f(n) = \omega(g(n)) \Rightarrow f(n) = \Omega(g(n))$
- $\lim_{n\to\infty} \frac{f(n)}{g(n)} = k > 0$ allora $f(n) = \Theta(g(n))$

Oppure si può utilizzare un'interpretazione intuitiva delle seguenti definizioni:

Funzioni	Numeri reali
$f(n) = O(g(n))$	$f \leq g$
$f(n) = o(g(n))$	$f < g$
$f(n) = \Omega(g(n))$	$f \geq g$
$f(n) = \omega(g(n))$	$f > g$
$f(n) = \Theta(g(n))$	$f = g$

Figura 1: Interpretazione intuitiva delle definizioni

1.3 Analisi degli algoritmi

Solitamente si analizzano 3 aspetti dell'algoritmo:

- **Caso ottimo**: ovvero la complessità computazionale in condizioni ottimali.
- **Caso pessimo**: ovvero la complessità computazionale in condizioni sfavorevoli.
- **Caso medio**: ovvero la complessità computazionale media sul possibile input.

Solitamente quando si sviluppa un algoritmo ci si interessa principalmente al caso **medio/pessimo**

1.4 Analisi Ammortizzata

Serve in caso di algoritmi molto complicati ed è un metodo per valutare il costo medio di una sequenza di operazioni a differenza di quella degli algoritmi che calcola su singola operazione. Ci sono due metodi principali per fare questo tipo di analisi:

- **Metodo dell'aggregazioni**: si determina un limite superiore al costo totale si una sequenza di n operazioni e dividiamo per n.
- **Metodo degli accantonamenti**: si immagina che ci sia un budget per ogni singola operazione e si accantona il 'resto' che viene usato nel caso il budget non basti per alcune operazioni. Per capire se la stima del budget è corretta alla fine si avrà un surplus positivo, in caso esso sia negativo allora bisogna stimare un budget più alto.

1.5 Equazioni di ricorrenza

Esprimono il valore di una funzione in termini dei valori su input più piccoli, ci sono due metodi per risolvere le equazioni di ricorrenza:

- **Metodo dell'iterazione**: è un approccio di tipo *brute force* che consiste nell'eseguire iterativamente l'algoritmo in analisi per dedurre il patter e modellizzarlo matematicamente. A questo punto con il caso base si trova il valore dell'icognita i e la si sostituisce nella modellizzazione in modo tale da trovare il costo computazionale dell'algoritmo.
- **Master Theorem**: funziona con equazioni di ricorrenza nella forma $T(n) = aT(n/b) + cn^{\beta}$ e quindi non puo' essere applicato a tutte le equazioni di ricorrenza (tipo quella di Fibonacci che chiama 2 volte se stessa). Il teorema dice che calcolando $\alpha = log_b(a) = \frac{log(a)}{log(b)}$ ci possono essere 3 casi:
  1. Se $\alpha > \beta$ allora $T(n) = \Theta(n^{\alpha})$
  2. Se $\alpha = \beta$ allora $T(n) = \Theta(n^{\alpha} log(n))$
  3. Se $\alpha < \beta$ allora $T(n) = \Theta(n^{\beta})$

2 Problema dell'ordinamento

Sono degli algoritmi che dato in input un array di numeri lo ordinano in modo crescente o decrescente. Ci sono due proprietà fondamentali di questi algoritmi:

1. **In place**: l'algoritmo riordina gli elementi nell'array in input e non ne usa di aggiuntivi
2. **Stabile**: valori con la stessa **chiave** (valore associato all'elemento che è sempre confrontabile con altri elementi) appaiono nell'array ordinato nello stesso ordine di partenza.

2.1 Algoritmi incrementali

Sono algoritmi più semplici che costruiscono l'ordinamento dell'array un elemento alla volta, allo stesso tempo però sono anche i meno efficienti

2.1.1 Selection Sort

Partendo da un elemento lo scambia con l'elemento minimo presente nel resto dell'array; questa operazione la si ripete per ogni elemento (tranne l'ultimo) andando sempre ad avere a sinistra dell'elmento in analisi un array ordinato. A causa dell'implementazione degli array la funzione di swap necessita di ulteriore memoria rendendo l'algoritmo $non - stabile$.

```
1:  function SELECTIONSORT(INT A[1, ··· , n])
2:      for i = 1, ··· , n − 1 do
3:          m = MIN(A, i)      ▷ Search min in A[i, ··· , n]
4:          if m ≠ i then
5:              SWAP(A, i, m)      ▷ Swap A[m] with A[i]
6:
7:  function MIN(INT A[1, ··· , n], INT i)
8:      for j = i + 1, ··· , n do
9:          if A[j] < A[i] then
10:             i = j
11:     return i
12:
13: function SWAP(INT A[1, ··· , n], INT i, INT j)
14:     tmp = A[i]
15:     A[i] = A[j]
16:     A[j] = tmp
```

Figura 2: Possibile implementazione del Selection Sort

La ricerca del minimo richiede di analizzare tutti gli elementi da $i+1$ a n ($\Theta(n)$), questa operazione va pero' ripetuta $\Theta(n)$ volte portando la complessità computazionale di questo algoritmo a $\Theta(n^2)$.

2.1.2 Isertion Sort

Partendo dal secondo elemento e andando verso l'ultimo si inserisce il valore nella parte di array a sinistra nell'ordine giusto scambiandolo con tutti i valori maggiori di esso. Per come vengono eseguiti i confronti l'algoritmo è stabile.

```
1:  function INSERTIONSORT(INT A[1, ··· , n])
2:      for i = 2, ··· , n do
3:          j = i
4:          while j > 1 and A[j] < A[j − 1] do
5:              SWAP(A, j, j − 1)
6:              j = j − 1
```

Figura 3: Possibile implementazione dell'Insertion sort

Nel caso ottimo il for viene eseguito n volte e basta quindi $\Theta(n)$; nei casi medio e pessimo invece avrà un costo pari a $\Theta(n^2)$.

2.2 Divide et impera

Viene dalla tecnica romana e consiste nel dividere il problema in problemi più semplici (*Divide*) che poi vengono risolti in maniera facile (*Conquer*) e poi le soluzioni vengono fuse per risolvere il problema generale.

2.2.1 MergeSort

Consiste nello dividere ricorsivamente a metà l'array per poi oridnare i sotto-array che diventano sempre piu' grandi ma allo stesso tempo ogni meta' e' ordinata; questo algorimto può essere implementato in modo stabile. E' pero' molto complicato implementare questo algoritmo in modo tale che sia anche in-place.

```
1: function MERGESORT(INT A[1, ··· , n], INT p, INT r)
2:     if p < r then
3:         q = ⌊(p + r)/2⌋
4:         MERGESORT(A, p, q)
5:         MERGESORT(A, q + 1, r)
6:         ▷ Here A[p, ··· , q] and A[q + 1, ··· , r] are sorted
7:         MERGE(A, p, q, r)
```

Figura 4: Esempio di implmentazione del MergeSort

```
1: function MERGE(INT A[1, ··· , n], INT p, INT q, INT r)
2:     LET B[1, ··· , r − p + 1] BE A NEW ARRAY
3:     i = p                                   ▷ Index over A[p, ··· , q]
4:     j = q + 1                               ▷ Index over A[q + 1, ··· , r]
5:     k = 1                                   ▷ Index over B[1, ··· , r − p + 1]
6:     while i ≤ q and j ≤ r do
7:         if A[i] ≤ A[j] then ▷ Condition A[i] ≤ A[j] makes the algorithm stable
8:             B[k] = A[i]
9:             i = i + 1
10:        else
11:            B[k] = A[j]
12:            j = j + 1
13:        k = k + 1
14:    ▷ If i ≤ q append A[i, ··· , q] to B[1, ··· , k − 1]
15:    while i ≤ q do
16:        B[k] = A[i]
17:        k = k + 1, i = i + 1
18:    ▷ If j ≤ r append A[j, ··· , r] to B[1, ··· , k − 1]
19:    while j ≤ r do
20:        B[k] = A[j]
21:        k = k + 1, j = j + 1
22:    ▷ Copy the sorted numbers from B[1, ··· , r − p + 1] to A[p, ··· , r]
23:    for k = 1, ··· , r − p + 1 do
24:        A[p + k − 1] = B[k]
```

Figura 5: Esempio di implmentazione della funzione Merge

Questo algoritmo ha complessità $\Theta(nlog(n))$

2.2.2 QuickSort

Si sceglie all'interno dell'array un valore detto *pivot* e si riordina l'array in modo tale che tutti i numeri più grandi siano nella parte destra dell'array e tutti i più piccoli sono a sinistra; a livello pratico si ha un contatore che tiene traccia dell'indice dell'ultimo elemento che deve stare a sdinistra del pivot e quando se ne incontra un altro si scambia con quello successivo. A questo punto si chiama ricorsivamente l'algoritmo nei due sotto array escluso il pivot per riordinare l'array. Nel caso ottimo il costo è $\Theta(nlog(n))$, nel caso pessimo $\Theta(n^2)$ mentre nel caso medio è $O(nlog(n))$. La complessità dipende molto da quale numero si scelga come *pivot* perche' la piu' sono bilanciati i due sotto array piu' e' efficiente l'algoritmo, quindi un'implementazione che sceglie il pivot in maniera furba può rendere l'algoritmo estramamente rapido. Nel caso studito si sceglie un numero a caso perche' rende molto improbabile ricadere nel caso pessimo o sfavorevole.

2.2.3 DualPivot Quicksort

Crea delle partizioni rispetto a due *pivot* e rende quindi meno probabile ricadere nel caso pessimo ma è anche abbastanza ibrido come algoritmo perchè a seconda delle situazioni può chiamare diversi algoritmi che lo rendono, in generale, molto efficiente; in media ha complessità $O(nlog(n))$.

2.2.4 Tim Sort

E' il più recente degli algoritmi che inizialmente fu introdotto in Python e si basa sull'idea che in un array ci siano già delle sottosequenze ordinate e da li usa il merge o insertion per sistemare le sequenze tra di loro. Come complessità ha costo, nei casi medio e pessimo, pari a $O(nlog(n))$ mentre nel caso ottimo $O(n)$.

2.3 Algoritmi di ordinamento in tempo lineare

Sono algoritmi che possono essere usati in alcune condizioni speciali perche' un qualsiasi algoritmo di ordinamento basato sulla comparazione di valori non puo' avere un'efficienza migliore fi $\Theta(nlog(n))$ nel caso pessimo.

2.3.1 Counting Sort

Questo algoritmo può essere usato solo per ordinare interi all'interno di un intervallo $[a, b]$, esso usa un array d'appoggio di grandezza $k = b - a + 1$ per contare quante volte un valore sta nell'array di partenza, riordina l'array senza doppione e riempio l'array di partenza con i numeri in modo ordinato.

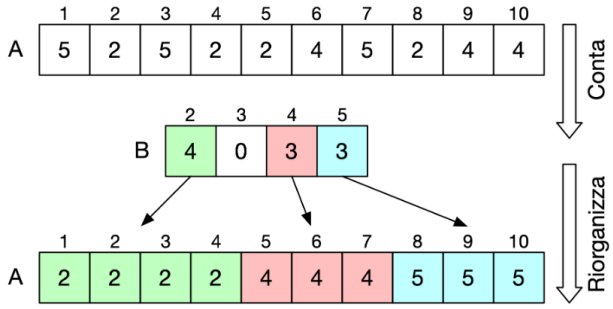


Figura 6: Esempio del funzionamento del CountingSort

```
1:  function COUNTINGSORT(INT A[1, ··· , n])
2:      a = MIN(A), b = MAX(A), k = b − a + 1
3:      LET B[1, ··· , k] BE A NEW ARRAY
4:      for i = 1, ··· , k do
5:          B[i] = 0
6:      ▷ Count
7:      for i = 1, ··· , n do
8:          B[A[i] − a + 1] = B[A[i] − a + 1] + 1
9:      ▷ Re-arrange
10:     j = 1
11:     for i = 1, ··· , k do
12:         while B[i] > 0 do
13:             A[j] = i + a − 1
14:             B[i] = B[i] − 1
15:             j = j + 1
```

Figura 7: Possibile implementazione CountingSort

Il costo di questo algoritmo è $\Theta(n + k)$ perchè se k è molto più grande di n o viceversa non è per nulla conveniente. Per rendere l'algoritmo stabile basta, una volta riempito l'array d'appoggio che conta gli elementi identici, bisogna eseguire le seguenti operazioni:

1. Ricodificare l'array d'appoggio per ottenere l'ultima posizione del dato elemento nell'array finale. Per farlo basta usare la formula:

```
V[j] = V[j-1] + V[j];
```

2. Riempire l'array finale partendo dall'ultimo elemento dell'array iniziale e metterlo nella posizione codificata dall'array d'appoggio:

```
A[n]; //given array
B[a, ..., b]; //support array alredy refactored
C[n]; //final array
for (int i = n-1; i >= 0: i--){
    C[B[A[i]]] = A[i];
    B[A[i]]--;
}
```

2.3.2 RadixSort

Questo algoritmo usa il counting sort come sotto-procedura infatti il RadixSort considera tutti gli interi con lo stesso numero di cifre e li riordina rispetto alla cifra meno significativa (attraverso il CountingSort), poi rispetto alla penultima e così via fino a quella più significativa. Il costo di questo algoritmo nel caso in cui **CountingSort** sia stabile (se no non funziona) è di $\Theta(d(n+k))$ con:

- d il numero massimo di cifre di una chiave
- k è il numero di possibili valori per una cifra

2.4 Riassunto costi degli algoritmi visti

	Caso ottimo	Caso medio	Caso pessimo
SelectionSort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
InsertionSort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
QuickSort	$\Theta(n \log n)$	$O(n \log n)$	$\Theta(n^2)$
CountingSort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$
RadixSort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$

	In place	Stabile
SelectionSort	Si	No
InsertionSort	Si	Si
MergeSort	No	Si
QuickSort	Si	No
CountingSort	No	Si
RadixSort	No	Si

- n = numero di elementi da ordinare
- d = numero massimo di cifre in una chiave
- k = ampiezza del range di valori chiave

Figura 8: Tabella riassuntiva dei costi degli algoritmi di ordinamento esaminati

3 Strutture di dati elementari

Descrivono come sono **logicamente organizzati** i dati e ne definisce le **operazioni** per cui non si hanno informazioni riguardo a **quali** dati sono memorizzati. Ci sono due parti principali delle strutture dati:

- **Prototipo**
  - Descrive possibili valori e operazioni
  - Nasconde i dettagli implementativi
  - Permette di implementare la struttura
  - Permette di capire come usare la struttura
- **Implementazione**
  - Realizzazione di una struttura dati in uno specifico linguaggio di programmazione
  - Non visibile all'utente
  - Può influire sui tempi di esecuzione

3.1 Liste

Struttura di dati in cui tutti gli elementi sono organizzati in modo sequenziale e deve supportare almeno le operazioni di **ricerca, inserimento e rimozione**. Esse si possono implementare sia con strutture primitive ovvero degli *array* che hanno il vantaggio di avere un tempo di accesso lineare ma hanno una grandezza fissa dipendete dallo spazio di memoria contigua libera, oppure attraverso *linked list* che non dipendono dalla quantità di memoria contigua libera ma hanno un tempo di accesso agli elementi dipendete dalla lunghezza della lista.

3.1.1 Linked List semplice

Ogni nodo punta al successivo quindi la si puo' scorrere in un solo verso, le operazioni hanno i seguenti costi:

- **Search**: e' implementata attraverso una semplice ricerca lineare per cui ha costi:
  - Ottimo: $O(1)$
  - Medio: $\Theta(n)$
  - Pessimo: $\Theta(n)$
- **Insert**: inserimento in testa alla coda:
  - Ottimo: $O(1)$
  - Medio: $O(1)$
  - Pessimo: $O(1)$
- **Append**: inserimento in coda, per cui e' necessario scorrere tutta la lista:
  - Ottimo: $\Theta(n)$
  - Medio: $\Theta(n)$
  - Pessimo: $\Theta(n)$
- **Delete**: simile all'operazione di search ma elimina l'elemento:
  - Ottimo: $O(1)$
  - Medio: $\Theta(n)$
  - Pessimo: $\Theta(n)$

3.1.2 Double Linked List

Sono come le Linked list semplici ma hanno un puntatore al nodo precedente quindi possono essere lette anche al "contrario". Nonostante ciò tutte le operazioni hanno lo stesso costo computazionale delle Linked List normali ma si deve usare più memoria per ogni singolo elemento.

3.1.3 Linked List circolari

Sono *Double Linked List* ma con la differenza che l'ultimo e il primo elemento si puntano a vicenda in modo che sia logicamente circolare, i costi invariati rispetto alle altre tranne per l'inserimento in coda che è costante.

3.1.4 Linked List con puntatori a testa e coda

Come una normale Linked List con la differenza che si ha un puntatore anche per la coda della lista per cui costi computazionali rimangono invariati tranne per l'inserimento in coda che è costante.

3.2 Pila

La struttura pila supporta due operzioni principali: il **push** (aggiunta di un elemento) e **pop** (eliminazione dell'elemento più recente). Questa struttura dati si può implementare sia attraverso un array che con una lista. Nel caso di uso di un array **statico** ci possono essere errori di underflow/overflow ma costi computazionali costanti, mentre con un **array dinamico** si puo' variare l'array ma avendo un costo medio di $O(n)$ per ognuna delle operazioni.

3.3 Code

Questa struttura dati supporta solo due operazioni principali: **enque** (aggiunta di un elemento in fondo) e **dequeue** (rimuove l'ultimo elemento della coda); in pratica si tartta di *Linked List* con operazioni limtate. Questa struttura dati puo' essere efficientemente implementata (ovvero le operazioni hanno costo costante) attraverso *Linked list circolari*, *Linked List semplici* o *array circolari*.

3.4 Alberi

Sono strutture dati **gerarchiche**, a differenza delle precedenti che sono dette *lineari*, definite da:

- **Nodi**
- **Archi** che connetono i nodi
- Esiste **un solo percorso** da un nodo all'altro

Un albero puo' essere **ordinato** sse i figli di ogno nodo sono ordinati (primo, secondo, ...) e **radicato** ovvero se uno dei suoi nodi puo' essere identificato come radice.

3.5 Alberi Binari

Sono alberi in cui ogni nodo ha al massimo 2 figli, dunque, potendo identificare *figlio destro* e *figlio sinistro*, e' per definizione **ordinato** . La **profondita'** di un nodo e' la lunghezza dell'unico percorso che lo collega con la radice, questo numero e' identico a quello degli archi che collegano nodo-radice mentre un **livello** e' l'insieme di tutti i nodi su uno stesso livello. Per **altezza** di un albero si intende la massima profondita' (per convenzione si considera -1 la profondita' di un albero vuoto), mentre il **grado** di un nodo e' il numero dei suoi figli. Un albero binario si dice **completo** sse ogni nodo ha due figli e un albero **perfetto** e' un albero completo con tutte le foglie alla stessa profondita'.

Proprieta' fondamentale degli alberi Ogni albero non vuoto ha esattamente $n-1$ archi, con n il numero di nodi dell'albero.

3.6 Algoritmi di visita

Sono algoritmi per visitare tutti i nodi di una struttura dati albero.

3.6.1 Visita in profondita'

La ricerca va il piu' in profondita' possibile prima di visitare i nodi nello stesso livello

- **Pre-order**: visita il nodo stesso poi l'albero sinistro e infine quello destro; la sua complessita' e' $\Theta(n)$.
- **Post-order**: visita l'albero sinistro poi quello destro e infine il nodo stesso; la sua complessita' e' $\Theta(n)$.
- **In-order**: visita l'albero sinistro poi il nodo stesso e infine quello destro; la sua complessita' e' $\Theta(n)$.

3.6.2 Visita in ampiezza

La ricerca viene eseguita livello per livello

- **Visita in ampiezza**: in pratica si fa una coda e ad ogni ciclo si toglie dalla coda il nodo e si mettono in coda i nodi in modo che ci sia una stampa per livello; la complessita' e' sempre $\Theta(n)$.

3.7 Come contare tutti i nodi di un albero?

Inizialmente si potrebbe pensare di implementare un algoritmo con la stessa idea di fondo della visita che al posto di stampare valori conta i nodi ma questo richiederebbe una complessita' pari a $\Theta(n)$, esiste dunque un modo di rendere la complessita' di questa operazione costante aggiungendo un campo ad ogni nodo che permette di memorizzare il numero di nodi sottostanti ad esso. Questo pero' rende piu'dispendiosa una qualsiasi modifica dell'albero.

3.8 Alberi binari di ricerca (BST)

Sono alberi radicati con vincoli sull'organizzazione delle chiavi: tutte le chiavi a sinistra di un nodo sono minori di esso mentre tutte quelle a destra sono maggiori; con questa organizzazione è possibile fare una ricerca binaria su questa struttura dati e le visita **in-order** permette di stampare in ordine tutti gli elementi dell'albero. Le operazioni fondamentali su questa struttura dati sono:

- **Search**: ritorna il nodo con chiave k, essa sfrutta la topografia della struttura infatti confrontando il valore cercato con la chiave del singolo nodo si sposta a destra nel caso $k > node.key$, al contrario si sposterà a sinistra. Questa operazione ha un costo nel caso pessimo pari a tutti i nodi (caso alberoi degenere).
- **Insert**: inserisce un nodo, anche questa operazione sfrutta la topologia della struttura dati ed e' simile alla search, nel caso pessimo, ovvero l'albero degenre, ha una complessita' $\Theta(n)$, mediamente il costo dipende pero' dall'altezza dell'albero.
- **Delete**: rimuove un elemento dalla struttura dati, questa operazione necessita pero' di alcune operazioni di supporto:
  - **Max**: ritorna il nodo massimo nell'albero ovvero lo percorre scegliendo sempre il figlio destro.
  - **Min**: ritorna il nodo minimo nell'albero ovvero lo percorre scegliendo sempre il figlio sinistro.
  - **Predecessor**: ritorna il nodo precedente a T in una visista in-order. Per quanto riguarda l'implementazione ci sono due casi distinti:
    - **Caso 1**: il nodo di cui si vuole trovare il precdecessore ha un figlio sinistro: il nodo predecessore e' il nodo con chiave massima nel sotto-albero sinistro. Per trovarlo e' necessario usare la funzione max nel sotto-albero sinitro: $pred = max(T.left)$

* **Caso 2**: il nodo di cui si vuole trovare il predecessore non ha un figlio sinistro; il nodo predecessore e' il primo antenato per cui il nodo di cui si sta cercando il predecessore e' nell'sotto-albero destro. Per trovarlo e' sufficiente ripercorrere l'albero verso l'alto (trmite il campo *parent*) fino a quando $N_pred = T.left$

– **Successor**: simmetrico a predecessor.

Anche nel caso della rimozione ci sono tre casi da considerare nell'implementazione:

– **Caso 1**: Il nodo da rimuovere e' una foglia, questo caso e' semplice perche' la regolarita' dell'albero non viene intaccata.

– **Caso 2**: Il nodo da rimuovere ha un solo figlio; in questo caso il figlio del nodo da rimuovere diventa il figlio del genitore del nodo da rimuovere.

– **Caso 3**: Il nodo da rimuovere ha 2 figli; in questo caso si copiano i dati del predecessore del nodo da rimuovere nel nodo stesso e si rimuove il nodo di cui sono stati copiati i dati.

A livello implementativo si usano 2 funzioni: la funzione **delete** che scambia i nodi nel caso 2 e 3 mentre la funzione **disconnect** che disconnette il nodo cfon controlli per casi particolari.

3.9 Riassunto

DIZIONARIO: RIASSUNTO DEI COSTI

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Albero Binario di Ricerca	$\Theta(\overline{h})$	$\Theta(h)$	$\Theta(\overline{h})$	$\Theta(h)$	$\Theta(\overline{h})$	$\Theta(h)$

- h = altezza dell'albero, $\overline{h}$ = altezza media dell'albero
  - $h = \Omega(\log n)$ e $h = O(n)$

Figura 9: Riassunto costi strutture viste fino a questo punto

3.10 Alberi AVL

Prendono il nome dai loro inventori, trattasi di BTS (alberi binari di ricerca) ma che siano sempre bilanciati in modo tale da avere un costo delle operazioni nel caso pessimo sempre di $O(log(n))$, per far cio' le operazioni di aggiunta e rimozione sono da modificare in modo da avere un albvero sempre bilanciato

3.10.1 Quando posso dire che un albero e' bilanciato?

Definiamo inizialmente il **fattore di bilanciamento** di un nodo ovvero la differenza tra l'altezza del suo sotto albero di sdinistra con quello di destra:

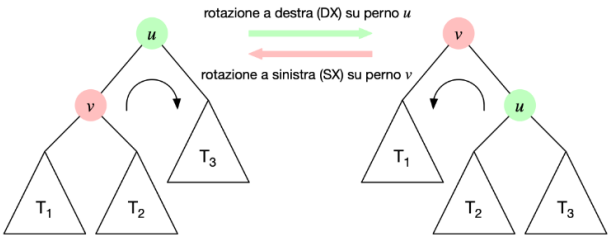
$$\beta(v) = altezza(v.left) - altezza(v.right)$$

A questo punto possiamo definire un **albero bilanciato** in altezza come un albero in cui ogni nodo la differenza di altezza tra i suoi sotto-alberi non e' mai maggiore di uno:

$$|\beta(v) \leq 1|$$

3.10.2 Operazioni basiche

1. **Search**: viene eseguita come una normale ricerca su un albero BTS
2. **Update height**: aggiorna l'altezza dell'albero, ogni nodo ha un suo paramentro che contiene l'altezza del sotto albero sottostante
3. **Balance factor**: aggiorna il fattore di bilanciamento di un nodo tramite il campo height dei suoi figli.
4. **Rotazioni**: esse sono fondamentali per ribilanciarte alberi AVL che in seguito a modifiche non sono piu' bilanciati. Si tratta di scegliere opportunamete un nodo perno e farlo diventare la radice, per mantenere l'albero sia binario che di ricerca bisogna fare si che il sotto-albero destro del nodo perno diventi il sotto albero di sinistra del padre, al contrario in caso di rotazione a destra:



- Preserva la proprietà di ordine dei BST
  - La chiave $u.key \geq v.key$ e maggiore o uguale di ogni chiave in T_1, T_2
  - La chiave $v.key \leq u.key$ e minore o uguale di ogni chiave in T_2, T_3
- Una rotazione semplice modifica solo il fattore di bilanciamento di u, v

Figura 10: Esempio e proprieta' delle rotazioni

Nel caso dunque che un albero AVL diventi sbilanciato si possono verificare 4 situazioni diverse:

- Sbilanciamento SS: $\beta(u) = 2$ e $\beta(u.left) \geq 0$
- Sbilanciamento DD: $\beta(u) = -2$ e $\beta(u.right) \leq 0$
- Sbilanciamento SD: $\beta(u) = 2$ e $\beta(u.left) = -1$
- Sbilanciamento DS: $\beta(u) = -2$ e $\beta(u.right) = 1$

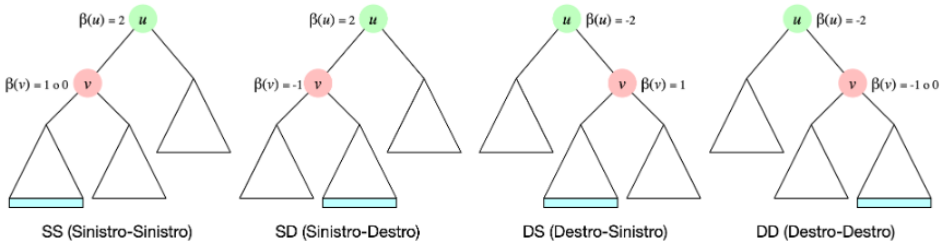


Figura 11: Casi di sbilanciamento degli alberi

A questo punto si illustrano i modi per ribilanciare gli alberi nelle varie situazioni:

- **SS**: si esegue una rotazione a destra con perno il nodo con $\beta(n) = 2$.
- **SD**: si esegue una rotazione a destra sul sottoramo sinistro in modo da avere uno sbilaciamento SS e poi si bilancia come un normale SS.
- **DD**: opposto al caso SS, dunque si opera una rotazione a sinistra con perno il nodo sbilanciato.
- **DS**: si opera una rotazione a destra sul sottoramo destro e poi si risolve come un DD

5. **Bilanciamento**: funzione molto semplice a questo punto perche' usa quelle sopra citate si per individuare il caso che risolverlo.
6. **Insert**: inserisce un nodo come in un normale BST ma alla fine chiama la funzione di bilanciamento dell'albero, al massimo puo' aver sbilanciato un nodo
7. **Delete**: anche in questo caso viene utilizzata la funzione dei BST ma questa procedura puo' comportare lo sbilanciamento di piu' nodi.

3.10.3 Riassunto

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Albero Binario di Ricerca	$\Theta(\overline{h})$	$\Theta(h)$	$\Theta(\overline{h})$	$\Theta(h)$	$\Theta(\overline{h})$	$\Theta(n)$
Albero AVL	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

- h = altezza dell'albero, $\overline{h}$ = altezza media dell'albero
- L'implementazione di un dizionario con Alberi AVL richiede unicamente la modifica dell'operazione di inserimento per evitare chiavi ripetute (esattamente come abbiamo discusso per gli Alberi Binari di Ricerca)
- Gli Alberi AVL assicurano un costo logaritmico per tutte le operazioni

Figura 12: Riassunto costi delle operazioni delle strutture visto fino ad ora

4 Tabelle HASH

L'idea dietro a questo tipo di struttra e' quella di riuscire a generalizzare l'indicizzazione in un array normalissimo in modo tale da avere un computazione che mediamente e' molto efficiente. Ci sono due principali implementazioni che si usano in casi diversi: le **tabelle ad indirizzamento diretto** che si usano quando il numero di chiavi effettivamente utilizzate e' simile a quello di tutte le chiavi possibili e le tabelle HASH che vengono usate quando il numero di chiavi utilizzate e' molto minore di quelle possibili.

4.1 Tabelle ad indirizzamento diretto

Sono implementate tramite array dalla dimensione statica, ogni elemento contiene sia una chiave per essere riconosciuta che un campo per il vlore in se. In realtà la chiave è la posizione dell'elemento all'interno dell'array (unica struttura dati a indirizzamento diretto), le operazioni basilari come *search, insert e delete* hanno un costo computazionale pari a $O(1)$ ma il vero problema e' la memoria che viene utilizzata: se le chiavi realmente utilizzate sono molte meno di quelle disponibili alllore c'e' un alto spreco di memoria.

4.2 Tabelle Hash

Con le tabelle HASH si cerca una soluzione al problema della memoria attraverso una **funzione di HASH** $h : U \to [0, ..., m-1]$ che partendo dal dato da memorizzare sono in grado di generare la chiave intera che corrisponde all'indirizzo all'interno dell'array. A questo punto sopraggiunge il problema delle **collisioni** ovvero come gestire il caso in cui due dati differente siano mappati nello stesso indirizzo.

4.3 Funzione Hash

Una buona funzione di hash soddisfa la proprieta' di **uniformita' semplice** ovvero c'e' un equiprobabilita' di generare ogni chiave perche' nel caso ci siano indici che vengono scelti piu' frequentemente come conseguenza si avranno sicuramente maggiori coillisioni. Essendo pero' questo irrealistico ci sono 3 assunzioni che vengono fatte riguardo ad una funzione hash:

1. Tutte le chiavi sono **equiprobabili** anche se non e' sempre vero.
2. La complessita' di calcolo della funzione hash e' $O(1)$ perche' se no costerebbe di piu' calcolare la funzione che cercare l'elemento all'interno dell'array.
3. Tutte le chiavi sono numeri interi e non negativi.

4.3.1 Metodo della divisione

Si restituisce il modulo tra il valore e la lunghezza della tabella, nel caso siano stringhe si convertono i singoli caratteri in valori numerici tramite la rappresentazione binaria. Questo metodo è molto efficiente ma dipende in modo molto stretto dalla scelta della lunghezza, infatti se la dimensione dell'array fosse 10 del valore si utilizzerebbe solo l'ultima cifra, mentre se la lunghezza fosse 2^p allora di un valore da mappare si utilizzerebbero i p bit meno significativi nella rappresentazione binaria.

4.3.2 Metodo delle moltiplicazioni

Usano la formula $h(k) = \lfloor m(kC - \lfloor kC \rfloor) \rfloor$ con

- m = numero di slot nell'array
- $0 < C < 1$
- $\lfloor x \rfloor$ = parte intera di x

C influenza l'uniformità di $h(k)$ ma il valore di m in questo caso non è critico.

4.3.3 Metodo della codifica algebrica

Si prende il valore in binario e la funzione di hash e' la seguente:

$$h(k) = (k_n x^n + k_{n-1} x^{n-1} + ... + k_1 x + k_0)$$

- k_n: corrisponde all'n-esimo bit della rappresentazione binaria
- x e' un valore costante

Alla fine si calcola il risultate e si fa il modulo con la grandezza dell'array. Questo medoto ha il vantaggio di dipendere da tutti i caratteri dell chiave ma e' molto costoso infatti non e' piu' costante ma $\Theta(log^2(k))$

4.3.4 Regola Di Horner

valupa il polinomio del metodo precedente in un punto e lo riscrive come

$$p(x) = a_0 + x(a_1 + x(...x(a_{n-1} + a_n x)))$$

Questo medoto permette di abbasare il costo a $\Theta(log(k))$ ma in generale essendo il numero di cifre di una chiave relativamente piccolo si puo' assumere come costante il tempo della codifica.

4.4 Problema delle collisioni

Non esiste un modo per eliminare del tutto le collisioni che sono soprendentemente molto probabili per cui e' necessario trovare un modo efficiente di gestire queste eventualita'. Ci sono due possibili tecniche per gestire questa problematica: il **concatenamento** e l'**indirizzamento aperto**.

4.4.1 Concatenamento

Lo slot $T(i)$ contiene il puntatore ad una lista concatenata che contiene tutti gli elementi con hash i, quando viene inserito un elemento esso e' inserito in testa alla lista mantenedo cosi' un costo costante. Nelle operazioni di ricerca o eliminazione il costo computazionale medio dipende dal numero di accessi che a sua volta dipende dalla distribuzione delle chiavi all'interno dell'array, per viene indicato come **fattore di carico** il rapporto tra il numero di elementi e la dimensione dell'array: $\alpha = n/m$. Dunque sotto l'assunzione di hashing uniforma semplice ogni slot ha α elementi e un ricerca senza successo o con successo ha costo medio $\Theta(1 + \alpha)$ per cui il fattore di carico ha un ruolo determinante per questa strategia.

4.4.2 Indirizzamento apero

L'idea alla base e' che se un solt e' gia' occupato se ne trova un altro che sia libero, la **sequenza di ispezione** che come parametro ha anche il **passo di ispezione** deve fornire una permutazione degli indici della tabella; un'altra criticita' e' la gestione della delete: eliminato un elemento non si puo' settare lo slot a null poiche' un eventuale ricerca, che passa per quella casella, si interromperebbe pensando che l'elemento non sia presente anche nel caso in cui la sequenza di ispezione dell'elemento cercato sia passata per l'elemento rimosso e sia andata avanti. Nel caso pessimo le tre operazioni fondamentali hanno costo $\Theta(m)$ con m dimensione della tabella, nel caso medio invece la complessita' dipende molto dalla strategia di ispezione:

- **Ispezione lineare**: $h(k, i) = (h'(k) + i) \ mod \ \ m$, ovvero data h' una normale funzione di hashing si va, ad ogni collisone e al crescere di i, a ispezionare sempre slot successivi. Questo rende il percorso utile con m percorsi possibili ma porta al **clusting primario** ovvero delle lunghe sotto-sequenze occupate tendono a diventare sempre piu' lunghe, uno slot vuoit preceduto da i solt pieni ha una probabilita' di rimepirsi di $(i + 1)/m$ quinid i tempi di calcolo crescono sempre di piu'.
- **Ispezione quadratica**: $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \ mod \ \ m, \quad c_1 \neq c_2$, questa ispezione pero' porta al **clustering secondario**: se due chiavi hanno $h'(k)$ uguale essi faranno le stesse visite

- **doppio hashing**: $h(k,i) = (h_2(k) + ih_2(k))\ mod\ \ m$, in questo caso si usano due funzioni di hashing in modo da evitare clutering primario e secondario ma $h_2(k) \neq 0$ perche' se no non cambia mai lo slot che si visita e ovviamente deve permettere di iterare su tutta la tabella.

A questo punto si puo' analizzare il caso medio dell'ispezione: anche in questo caso si usa il fattore di carico che si assume $\alpha < 1$ poiche' ci sono piu' slot che elementi da memorizzare, inoltre si suppone si usi un hashing uniforme semplice e che le permutazioni siano tutte equiprobabili. Dunque una ricerca senza successo necessita di un numero massimo di $\frac{1}{1-\alpha}$, mentre nel caso di successo le operazioni necessarie sono al massimo di $\frac{1}{\alpha}\ ln(\frac{1}{1-\alpha})$. Generalmente si cerca di tenere un fattore di carico $\alpha < 0.75$

4.5 Riassunto

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Albero Binario di Ricerca	$\Theta(\overline{h})$	$\Theta(h)$	$\Theta(\overline{h})$	$\Theta(h)$	$\Theta(\overline{h})$	$\Theta(n)$
Albero AVL	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
Tabelle Hash + con	$O(1)$	$\Theta(n)$	$O(1)$	$\Theta(n)$	$O(1)$	$\Theta(n)$
Tabelle Hash + ind	$O(1)$	$\Theta(m)$	$O(1)$	$\Theta(m)$	$O(1)$	$\Theta(m)$

- h = altezza dell'albero, $\overline{h}$ = altezza media dell'albero
- n = numero di elementi nella struttura dati, m = dimensione della Tabella Hash

Figura 13: Riassunto costi delle strutture viste fino ad ora

5 Heap binari

L'utilizzo di questa struttura dati e' rilegata alle situazioni in cui quando si vogliono estrarre i dati essi devono essere o il minimo o il massimo dato memorizzato. Per implementare questa struttura si usano degli alberi binari **quasi perfetti** con particolari proprieta' dei nodi.

5.1 Albero binario quasi perfetto

Un albero binario risulta **perfetto** se ogni livello dell'albero contiene il massimo dei nodi possibili e il numero di nodi risulta essere $2^{h+1} - 1$ per cui l'altezza sara' $h \approx log(N)$, con N il numero di nodi. A questo punto si definisce **albero binario quasi perfetto** come un albero che e' perfetto in tutti i livelli tranne nel finale, le foglie devono essere compattate a sinistra e solo un nodo all'interno dell'albero deve avere grado minore di 2.

5.2 Alberi heap

5.2.1 Max-heap

E' un albero binario quasi perfetto in cui ogni nodo padre contiene un valore che e' maggiore di quello del figlio, dunque la radice ha il valore massimo.

5.2.2 Min-heap

E' un albero binario quasi perfetto in cui ogni nodo padre contiene un valore che e' minore di quello del figlio, dunque la radice ha il valore minimo.

5.3 Array Heap

E' il modo piu' efficace di implementare la struttura logica ad albero: a livello logico si sta usando un albero ma in realta' a livello pratico si usa un array. Per riempire l'array bisogna fare una visita in ampiezza in modo tale da riempire l'array in ordine rispetto ai livelli dell'albero dunque il primo elemento dell'array contiene il valore piu' alto inoltre valgono le relazioni che il figlio sinistro di un elemento i si trova in posizione $2i$ mentre il figlio destro si trova in posizione $2i + 1$; se invece si volesse trovare il padre del nodo la sua posizone e' $\lfloor i/2 \rfloor$. A questo punto si possono descrivere le varie operazioni possibili su questa struttura:

- **findMax**: basta guardare il primo elemento nell'array, il costo e' $O(1)$

- **fixHeap**: sistema l'albero quando il massimo viene eliminato supponendo pero' che i due sotto-alberi siano quai perfetti. Inizialmente la radice diventa il figlio con valore maggiore, a questo punto il "buco" passa al livello piu' basso e si tratta di risolvere lo stesso problema in maniera ricorsiva fino a quando il "buco" viene sistemato. Questa operaqzione e' stata descritta come post-delete ma si puo' utilizzare anche nel caso di heapify. Il costo computazionale al peggio e' un confronto per ogni livello per cui $\Theta(log(n))$ per cui il caso medio e' $O(log(n))$.
- **heapify**: partendo da un array senza alcun ordine si crea un array heap, viene implemntato con una funzione ricorsiva che prima si chiama sui due sotto-alberi (grazie alle formule viste prima) e poi chiama fixHeap per sistemare l'alberone. Per stimare il costo computazionale si usa l'equazione di ricorrenza della funzione ricorsiva: $T(n) = 2T(n/2) + log(n)$. Col master theorem non si reisce a stimare bene quindi si calcola un limite superiore tramite la disequazione $2T(n/2) + log(n) < 2T(n/2) + n^{\frac{1}{2}}$ per cui col master theorem posso stimare una complessita' di $O(n)$
- **deleteMax**: rimuove l'elmento massimo mantenendo le proprieta' dell'array heap, per implementarlo basta rimuovere il primo elemento e mettere in prima posizione l'ultimo elemento poi far partire il fixHeap.

5.4 Heap Sort

L'idea e' quella di costruire, a partire da un generico array, un max-heap tramite heapify poi si estrae l'elemento piu' grande come descritto prima e si va avanti cosi' fino a quando non sara' vuoto e avendo cosi' ordinato l'array, il costo computazionale a questo punto risulta essere pseudolineare: $O(nlog(n))$.

6 Coda con priorità

E' una struttura dati che contiene delle coppie chiave-elemento che devono però essere confrontabili. Le operazioni possibili con questa struttura dati sono:

- **findMin**: restituisce l'elemento associato alla chiave minima che è l'elemento nella radice. Costo costante.
- **insert**: inserisce una coppia nella struttura dati. Viene aggiunto l'elemento nel primo spazio disponibile e poi tramite la funzione ausiliaria muoviAlto si sistema l'elemento. Costo dell'operazione $O(log_d(n))$
- **delete**: rimuove un elmento dalla coda indipendentemente dalla sua chiave, supponendo un accesso diretto a tale elemento. Dunque si sostituisce il nodo da rimuovere che l'ultima foglia e poi si usano le due funzioni ausiliarie per sistemarlo propriamente (solo una delle due entrerà in azione). Costo complessivo $O(d\ log_d(n))$.
- **deleteMin**: rimuove l'elemento associato alla chiave minore.
- **increaseKey**: rimpiazza la chiave di un elmento con una chiave maggiore, per implemntarlo si cambia la chiave e poi si usa muoviBasso. Costo di $O(d\ log_d(n))$
- **decreaseKey**: opposto di increaseKey. In questo caso si decrementa la chiave e poi si esegue un muoviBasso per sistemare la struttura. Costo di $O(log_d(n))$

Per implementare le code con priorità ci sono due principali soluzioni ma quella che studieremo è il **D-Heap**.

6.1 D-Heap

Estendono il concetto di albero di max/min-heap e si basano su di essi per la loro implementazione, in questo caso però è al contrario rispetto a quello visto in precedenza perchè si usa un min-heap per cui ogni padre è il minore dei figli, in questo modo nella radice dell'albero sarà presente la chiave minima. Anche in questo caso si ha una corrispondenza tra l'altezza dell'albero d-heap e il numero n di nodi ovvero $O(log_d(n))$; come negli alberi heap a livello di memoria anche per gli alberi d-heap si usa un array, essendo non binari gli alberi ma $d-ari$ le relazioni tra padre e figli sono:

- **Ultimo figlio**: $(i * d) + 1$
- **Primo figlio**: $((i-1) * d) + 2$
- **Padre del nodo**: $\lceil (i-1)/d \rceil$

Ci sono anche alcune operazioni ausiliarie alle altre procedure ovvero:

- **muoviAlto**: che sposta in alto un valore che è mal posizionato, costo pari a $O(h)$.
- **muoviBasso**: fa il contrario di muoviAlto portando in basso l'elemento. Il costo è pari a $O(dh)$.

6.2 Riassunto D-Heap

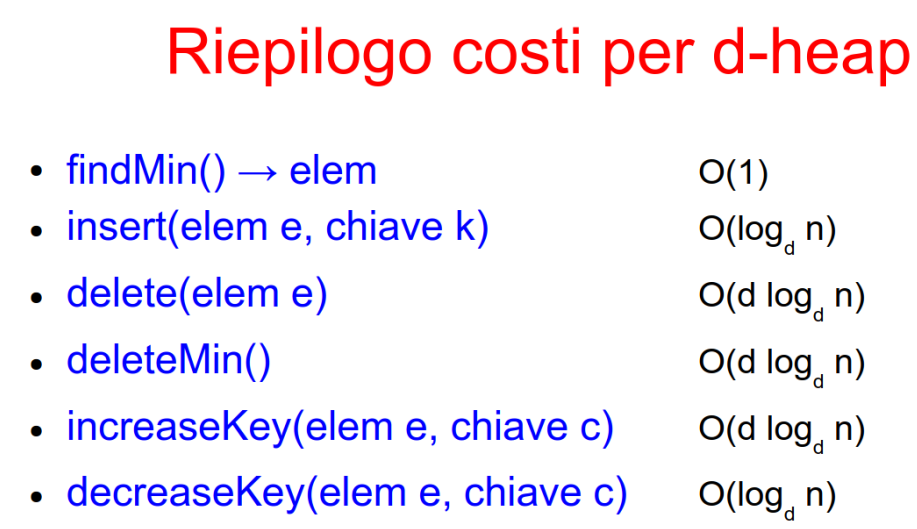


Figura 14: Riassunto costi delle procedure relative ai d-heap

7 Union Find

Questa struttura dati viene impigata quando si ha la necessità di gestire insiemi disgiunti di oggetti, per cui le operazioni fondamentali saranno la creazione di un singoletto, unione di due insiemi e l'identificazione a quale insieme appartiene un dato elemento, piu' precisamente chi e' il rappresentante del tale elemento.

- **makeSet**: crea l'insieme singoletto con il dato elemento
- **find**: restituisce il rappresentante dell'insieme a cui appartiene il dato elemento
- **union**: unisce due insiemi di cui sono stati forniti i rappresentanti

7.1 QuickFind

Questi algoritmi utilizzano alberi con altezza 1 per implementare la struttura dati Union-find: ogni elemento e' una foglia e il rappresentante e' la radice (quindi il rappresentante e' un duplicato essendo sia radice che foglia).

- makeSet: crea un albero con l'elemento che e' sia radice che foglia.
- find: dato l'elemento restituisce il proprio padre, dunque questi alberi hanno i puntatori invertiti rispetto a come si implementano di solito.
- union: e' la piu' dispendiosa perche' prende il secondo albero passato (la radice) e cambia tutti i puntatori dei figli facendoli puntare al nuovo padre, nel caso pessimo deve svolgere questa operazione $n-1$ volte.

7.1.1 Euristica sul peso (union)

Grazie a questo algoritmo si sceglie quale rappresentante mantenere, tra i due dati in input, in base a quanti figli essi hanno (basta riservare un campo specifico nel rappresentante e sommare i numeri ad ogni invocazione). Dunque nel caso pessimo si dovranno eseguire $\frac{n}{2}$ operazioni e attraverso un'analisi ammortizzata il costo risulta essere $O(log(n))$

7.2 QuickUnion

Questi algoritmi utilizzano alberi generali per implementare la struttura dati Union-find: in questo caso gli elemnti possono essere anche nodi per cui il rappresentante e' il nodo che non ha padre.

- makeSet: crea un albero con l'elemento (no duplicati tra radice e foglia).
- find: scorre l'albero andando verso l'alto fino a quando non arriva alla radice, nel caso pessimo deve quindi scorrere tutti gli elementi.
- union: dati i due insiemi fa puntare il rappresentante del secondo al rappresentante del primo, per cui ha costo costante.

7.2.1 Euristica "union by rank" (find)

Avere alberi molto alti rende molto inefficiente l'operazione di find quindi memorizzando nelle radici il proprio rank (altezza) si puo' sistamare l'albero opportunamente in modo tale che non degeneri in una lista. Grazie a questo approccio il costo scende a $O(log(n))$.

7.3 Riassunto Union Find

	QuickFind	QuickUnion	QuickFind eur. peso	QuickUnion eur. rank
makeSet	O(1)	O(1)	O(1)	O(1)
union	O(n)	O(1)	O(log n) amm.	O(1)
find	O(1)	O(n)	O(1)	O(log n)

Figura 15: Riassunto costi delle procedure relative alla struttura Union Find

8 Tecniche algoritmiche

- **Divide-et-impera**: un problema viene diviso in sotto-problemi da risolvere ricorsivamente.
- **Greedy**: Ad ogni passo si fa sempre la scelta che in quel momento eà a migliore e non si ripercorrono mai le scelte fatte in precedenza.
- **Programmazione dinamica**: la soluzione viene costruita con un approccio bottom-up a partire da un insieme di sotto-problemi.

8.1 Torre di Hanoi

E' un gioco matematico in cui ci sono 3 pioli, n dischi in un piolo in ordine crescente e l'obiettivo e' trasferire l'intera pila su un altro piolo. Le regole sono che si puo' spostare un disco alla volte e che un disco deve essere impilato solo sui dischi piu' grandi di esso. L'analisi del costo computazionale si puo' fare tramite l'equazione di ricorrenza $T(n) = 2T(n-1) + 1$ e usando la tecnica dell'iterazione si arriva ad avere un costo totale pari a $O(n^2)$

8.2 Problema del resto

Dato un numero intero positivo R bisgona generare qual è il minimo numero di monete che permette di raggiungere quel resto. In questo caso l'approccio greedy e' quello che funziona meglio: ad ogni passo prendo il numero massimo di moneta piu' grande possibile e poi procedo con il taglio piu' piccolo e la rimanente parte di resto, questo sapendo che nel peggiore dei casi copriro' l'eventuale rimanenza con le monete da 1c; per cui questa soluzione funziona solo con **sistemi monetari canonici** infatti, per esempio, con monete = [4, 3, 1] e resto 6 questo approccio fornisce come soluzione 4, 1, 1 quando la soluzione ottimale sarebbe 3, 3.

8.3 Problema dell'imbianchino

E' un problema imile a quello del resto: si vuole imbiancare una certa parete per cui servono L litri di vernice e vuole comprare la giusta quantità di barattoli per avere L litri. Come visto prima l'approccio greedy non e' efficace in tutte le situazioni per cui si puo' usare la tecnica *divide-et-impera* che a livello logico costruisce un albero sondando ogni possibile combinazione di barattoli prendibile e scegliendo quella che ne richiede di meno. Con questo approccio pero' molti calcoli vengono fatti inutilmente per cui conviene utilizzare la programmazione dinamica per risolvere il problema: si utilizza un array $[0, ..., L]$ con le soluzioni migliori ad ogni grandezza minore di L ed ogni elemento si calcola facendo il minimo tra tutte le combinazioni possibili andando anche a vedere le soluzioni gia' ottenute per evitare di rifare calcoli.

8.4 Problema dello zaino

Si ha uno zaino con una capienza di peso massima P e si hanno a n oggetti con proprio peso e valore, l'obiettivo è trovare il valore maggiore che può essere portato nello zaino. In elenco alcune soluzioni sbagliate seguendo un approccio **greedy**:

- Ad ogni passo scelgo sempre l'oggetto disponibile con peso maggiore che si può mettere nello zaino finchè non finisce lo spazio oppure nessun oggetto può essere inserito.

- Si calcola un valore specifico dell'oggetto (valore/peso) e si prende sempre quello piu' conveniente possibile.

Entrambe le situazioni proposte sopra non sono efficaci perchè non trovano sempre la combinazione migliore per cui è necessario un approccio che si basa sulla programmazione dinamica. La programmazione dinamica basa la soluzione sull'utilizzo di 2 matrici con righe il numero di oggetti e colonne lo spazio dello zaino da 0 a P; ogni casella viene riempita nel modo più efficiente possibile a seconda dell'elemento che ha lo stesso indice della riga. A livello matematico ci sono 4 casi:

1. L'oggetto è maggiore della capienza dello zaino.
2. Lo zaino può contenere l'oggetto e inserirlo massimizza il valore.
3. Lo zaino può contenere l'oggetto ma inserirlo non massimizza il valore.

A livello di formule si tratta di: $max\{A[i-1,j], A[i-1,j-p_oggetto]+valore_oggetto\}$ e alla fine si dovrà prendere la cella con il valore maggiore (ovvero quella più in basso a sinistra). La seconda matrice, delle stesse dimensioni della prima, contiene booleani in base a se l'oggetto della riga è stato inserito nello zaino dalla tale capacità. A questo punto per trovare la soluzione al problema originario si parte dall'ultima casella e si torna verso in su secondo la formula $A[i-1, j-p_oggetto]$ solo nel caso in cui si sia trovato True se no si torna alla casella sopra.

8.5 Esempi di utilizzo metodo Divide et Impera

8.5.1 Moltiplicazioni di interi di grandezza arbitraria

La maggior limitazione delle moltiplicazione è il max_int che dopo fa sballare tutto quindi sono necessari algoritmi per moltiplicare tra di loro numeri molto grandi. Inizialemnte si può pensare di implementare un algoritmo basato su come si fanno le moltiplicazioni in colonna ma esso ha un costo computazionale di $O(n^2)$ per cui è necessaria una versiona più veloce, attraverso il metodo *divide et impera* questo algoritmo può essere implementato inizialmente con una scissione a metà dei numeri:

$$X = X_1 * 10^{n/2} + X_0$$
$$Y = Y_1 * 10^{n/2} + Y_0$$
$$X * Y = (X_1 * 10^{n/2} + X_0)(Y_1 * 10^{n/2} + Y_0) = (X_1 * Y_1) * 10^n + (X_1 * Y_0 + X_0 * Y_1) * 10^{n/2} + X_0 * Y_0$$

A questo punto l'equazione di ricorrenza è c_1 per il caso base e $4T(n/2) + c_2 n$ altrimenti per cui attraverso il Master Theorem si trova una complessità di $O(n^2)$ per cui non c'è alcun miglioramento rispetto a quello normale. Per migliorare si riscrivono gli addendi in modo furbo:

- $P_1 = (X_1 + X_0) * (Y_1 + y_0)$
- $X_1 * Y_1$
- $X_0 * Y_0$
- $X * Y 0 P_2 * 10^n + (P_1 - P_2 - P_3) * 10^{n/2} + P_3$

Quindi sono necessarie 3 moltiplicazioni che generano una lunghezza dell'output di $O(n/2)$ cifre quindi l'equazione di ricorrenza di prima cambia in $3T(n/2 + c_2 * n)$ arrivando a $O(n^{log_2(3)}) \approx O(n^1.59)$

8.5.2 Sottovettore non vuoto di valore massimo

Il problema prevede di trovare un sottovettore ad un array dato (quindi elementi contigui) la cui somma sia massima. Il motodo *brute force* che calcola il valore di ogni combinazione ha un costo di $O(n^2)$ che puoò essere migliorato mantenendo in una variabile il valore della sommatoria precedente portando ad un costo $O(n^2)$. Con il metodo *divide et impera* l'idea è quella di trovare un pivot e calcolare il sottovettore massimo a destra e a sinistra di esso e poi vedere qual è il massimo possibile a cavallo con il pivot. Questo problema è più semplice perchè a per il sotto array a cavallo basta prendere il massimo a destra, a sinistra e sommare l'elemento a cavallo e infine ritornare il massimo tra solo destra, a cavallo, solo a sinistra. L'equazione di ricorrenza è dunque $T(n) = 2T(n/2) + n$ con un costo totale di $O(n\ logn(n))$.

8.6 Esempi di utilizzo metodo Greedy

8.6.1 Problema di scheduling

Dati dei "job" da eseguire con un certo tempo di esecuzione l'obiettivo è trovare l'ordine (scheduling) che dia il **minimo tempo di attesa medio** ovvero ogni quanto un nuovo job è iniziato. Per questo motivo l'ordine migliore è quello che mette in ordine dal più corto al più lungo dunque l'approccio greedy è il migliore.

8.6.2 Problema della compressione (codifica di Huffman)

Data una squenza di caratteri essa deve essere salvata in una memoria digitale quindi codificata e poi deve esssere anche possibile decodificarla, questo processo viene eseguito tramite una tecnica detta **codifica dei caratteri** quindi "il carattere x è rappresentato dalla sequenza c". Data questa codifica è necesario però decodificarla attraverso un semplice decodifica sequenziale. Il problema da risolvere è trovae la codifica che dia la codifica più corta possibile. La soluzione migliore è una **codifica a lunghezza variabile** che si basa sulla percentuale di apparizione del carattere: a caratteri che appaiono più spesso associo una codifica più corta e man mano aumento mantenendo una **code a prefisso** ovvero che nessun codice non può essere il prefisso di un altro codice in modo tale che appena si individua un carattere si sa che può essere solo quello. Per decodificare si può usare la struttura dell'albero binario fatto in modo che ci sia l'altezza media per nodo minore possibile per far si che il cammino medio sia il più piccolo possibile per essere più veloce possibile. Per costruire l'albero si usa l'**algoritmo di Huffman**: si costruisce inizialmente una losta ordinata di nodi in modo crescente rispetto a quante volte appaere il carattere nel file, successivamente si uniscono le foglie a coppie con un padre la cui etichetta ha la somma delle occorrenze ma ad ogni creazione di padre esso si mette nella lista ed essa viene riordinata e si ricrea un altro padre fino a quando non si è ottenuta una radice ed ecco ottenuto l'albero più efficiente possibile con costo computazionale $O(n\ log(n))$.

8.7 Esempi di utilizzo di programmazione dinamica

E' una tecnica iterativa che affronta i problemi con un approccio *bottom-up* e che è molto vantaggiosa quando si tratta di risolvere sottoproblemi ripetuti, comporta sempre l'appoggio ad una struttura dati ausiliaria.

8.7.1 Numero di Fibonacci, tecnica della Memorization

Questa tecnica usa una struttura *cache* che memorizza i valori che sono già stati calcolati rendendo così tutto più veloce perchè si va sempre a vedere se il numero è stato calcolato.

8.7.2 Sottovettore di valore masismo

La soluzione ragione per step: progressivamente trova il sottovettore massimo per l'i-esima cella del vettore che la contiene e per riempire la i-esima cella si fa $max(A[i-1]+S[i], S[i])$ con S l'array in input e A l'array ausiliario. Per evitare di dover cercare nell'array di appoggio il valore massimo si memorizza in una variabile l'indice del massimo e quindi il costo computazionale totale risulta essere di $\Theta(n)$. Per tagliare il costo di di memoria si possono usare delle variabili su cui ci si può appoggiare.

8.7.3 Seam Carving

Il problema vuole ridimensionare un'immagine tagliando solo delle cuciture di pixel removibili, i "pixel removibili" sono quelli con valore assegnatoli, tramite algoritmi non studiati, minore. Essendo l'immagine iniziale una matrice anche la struttura dati di appoggio sara' la medesima delle medesime dimensioni. Per quanto riguarda l'algoritmo in se' si tratta di stabilire, partendo dall'ultima riga, qual e' il valore del percorso piu' "basso" che parte dalla cima. Ricorsivamente si va a calcolare lo stesso per le 3 caselle adiacenti sopra e una volta riempita la matrice di appoggio con gli opportuni valori per trovare la cucitura da eliminare basta trovare il minimo nell'ultima riga e da li' risalir fino alla cime con un approccio greedy.

8.7.4 Distanza di Levenshtein

E' il problema che affrontano i controllori ortografici quando suggeriscono parole simili come correzioni a quelle digitate e la loro somiglianza si basa sul concetto di *edit distance* ovvero il numero di operazioni necessaria per trasformare una stringa T in una stringa S. Le uniche trasformazioni ammesse sono:

- Lasciare immutato il carattere → costo 0
- Cancellare un carattere → costo 1
- Inserire un carattere → costo 1
- Sostituire un carattere con uno differente → costo 1

Ovviamente la conversione viene operata carattere per carattere sulla stringa T. Per definizione la **distanza di Levenshtein** tra $S[1..n]$ e $T[1..m]$ e' il costo minimo (riferito a quelli esplicitati prima) tra tutte le sequenze di *operazioni di editing* che trasformano S in T. Per calcolare la soluzione si ricorre ad una matrice di appoggio in cui ci si segna quante operazioni devono essere effetuate (almeno) per trasformare la sequenza $L[0..i][0]$ in $L[0][0..j]$ in pratica si fa un confronto con le tre caselle precedenti (quella a sinistra, quella sopra e quella sopra a sinistra) e si guarda il carattere nella riga i e quello nella colonna j, se sono:

- **Uguali**: si prende il minimo tra casella a sinistra + 1, casella sopra + 1 e casella in alto a sinistra
- **Diversi**: si prende il minimo tra le tre caselle precedenti e si incrementa di 1

9 Grafi

Sono strutture dati che permettono di memorizzare quanto i diversi elementi si interfacciano tra di loro, i problemi che affronteremo sui grafi sono la visita (in ampiezza e in profondità), alberi di copertura minimi e cammini minimi (in caso di singola sorgente e fra tutte le coppie). Ci sono due definizioni di grafo che distinguono i due tipi:

- **Grafo orientato**: è una coppia di insiemi, il primo è quello dei vertici e l'insieme delle relazioni tra i vertici. Viene definito **cappio** un nodo in relazione con se stesso.
- **Grafo non orientato**: gli archi (ovvero le relazioni tra grafi) sono non ordinate (quindi non c'è una direzione) e i cappi non sono ammessi.

Ci sono alcune definizioni che sono itili in questa struttura: **incidenza** che, in un grafo orientato, indica che partendo da un vertice si può andare nel secondo mentre se due nodi sono **adiacenti** signigfica che c'è una relazione tra i due (ovvero ci si può arrivare). Le operazioni di base che il grafo deve supportare sono:

- NumVertici
- NumArchi
- grado (vertice) ovvero quanti collegamenti ha
- archiIncidenti
- estremi (arco)
- sonoAdiacenti (vertice, vertice)
- aggiungiVertice
- aggiungiArco
- rimuoviVertice
- rimuoviArco

9.1 Implementazione grafo non orientato, matrice di adiacenza

Si usa una matrice con tutti gli elementi sia nelle righe che nelle colonne e si mette 1 se due elementi sono collegati 0 se non lo sono. Quella che risulta è una matrice simmetrica con diagonale tutta 0 per l'assenza cappi. Lo spazio necessario è $\Theta(|V|^2)$. I costi delle operazioni sono:

- grado (vertice) $\rightarrow \Theta(n)$
- archiIncidenti $\rightarrow \Theta(n)$
- sonoAdiacenti (vertice, vertice) $\rightarrow \Theta(1)$
- aggiungiVertice $\rightarrow \Theta(n^2)$
- aggiungiArco $\rightarrow \Theta(1)$
- rimuoviVertice $\rightarrow \Theta(n^2)$
- rimuoviArco $\rightarrow \Theta(1)$

9.2 Implementazione grafo non orientato, lista di adiacenza

L'idea è quella di avere un array degli elementi che sono inizi di liste che contengono gli elementi a cui sono adiacenti, lo spazio di memoria è $\Theta(|V| + |E|)$ con V vertici e E le relazioni

- grado (vertice) $\rightarrow \Theta(\gamma(v))$
- archiIncidenti $\rightarrow \Theta(\gamma(v))$
- sonoAdiacenti (vertice, vertice) $\rightarrow \Theta(min\{\gamma(x), \gamma(y)\})$
- aggiungiVertice $\rightarrow \Theta(1)$
- aggiungiArco $\rightarrow \Theta(1)$
- rimuoviVertice (vertice) $\rightarrow \Theta(numero\ archi)$
- rimuoviArco (arco) $\rightarrow \Theta(\gamma(x) + \gamma(y))$ con l'arco $= \{x, y\}$

9.3 Implementazione grafo orientato, matrice di adiacenza

Uguale a quella con il non ordinato ma non è più simmetrica perchè in questo caso l'ordine di relazione conta e ci sono cappi.

9.4 Alcune definizioni

1. **Grafi pesati**: sono grafi in cui ogni arco ha un peso/costo, dunque per determinare il costo si utilizza un'opportuna funzione che, per comodità, restituisce ∞ quando non c'è una relazione tra i due elementi. Questa implementazione riguarda la matrice di adiacenza mentre nella lista di adiacenza si aggiunge un campo al singolo nodo.

2. **Grado**: dato un grafo non orientato il grado di un vertice è il numero di archi del dato vertice. Nel caso di grafo orientato c'è la distinzione in **grado entrante** e **grado uscente** ovvero il numero di archi in arrivo e in uscita.

3. **Cammino**: percorso che collega due vertici e se esiste si dice che w è raggiungibile da v tramite il cammino v. Il cammino prende in considerazione l'orientamento dei collegamenti nel caso del grafo orientato.

4. **Grafi connessi**: nel caso del grafo non orientato esso è connesso se esiste un cammino che da ogni vertice arriva ad ogni altro vertice. Nel caso del grafo orientato esso è **fortemente connesso** se esiste un cammino da ogni vertice ad ogni altro vertice mentre si dice **debolmente connesso** se l'equivalente non orientato del grafo orientato è connsesso.

5. **Ciclo**: cammino che parte da un elemento e attraversa un grafo ritornandoci. Nel caso di grafo orientato bisogna fare almeno 1 passo mentre in quelli non orientati almeno 3 passi. Un **ciclo semplice** è un cammino che non ritorno su nessun nodo già attraversato, mentre il **ciclo complesso** ha dei nodi ripetuti nel cammino.

6. **Grafi aciclici**: un grafo non orientato è aciclico se è senza cicli semplici mentre quello orientato lo è solo se non ci sono cicli.

7. **Grafo completo**: un Grafo non orientato è completo se ha un arco per ogni possibile vertice, per calcolare il numero di archi necessari a ciò si può usare la formula: $\frac{|V|*(|V|-1)}{2}$

8. **Albero**: un **albero libero** è un grafo non orientato connesso e aciclico, mentre se chiamiamo uno dei vertice radice otteniamo un **albero radicato**

9.5 Algoritmi di visita dei grafi

Dato un grafo ed un vertice (**sorgente**) l'obiettivo è visitare ogni possibile vertice del grafo raggiungibile dalla sorgente.

9.5.1 Visita in ampiezza (BFS)

Visita i nodi espandendo la frontiera tra nodi scoperti e nodi da scoprire. Alla fine dell'esecuzione ha creato un albero radicato in s (il nodo di partenza) che contiene tutti i nodi che si posso raggiungere da esso. Durante l'esecuzione un nodo puo' essere: **inesplorato** ovvero non ancora incontrato, **aperto** ovvero quando viene incontrato la prima volta oppure **chiuso** ovvero il vertice e' stato esplorato completamente. Per quanto riguarda l'implementazione essa si basa su il campo *mark* di ogni nodo che inizialmente setta in tutti a $false$ e poi grazie ad una coda esplora apre un nodo, mette in coda tutti i nodi nonm aperti raggiungibili da quello analizzato, chiude il nodo e continua con il successivo nodo in coda. Per quanto riguarda la creazione dell'albero ad ogni adiacente che viene aperto si riempie il campo *parent* con il nodo che si sta analizzando. Alla fine viene ritornato l'albero. Per quanto riguarda la complessità computazionale nel caso di *liste di adiacenza* e' $O(n+m)$ mentre nel caso della *matrice di adiacenza* e' di $O(n^2)$, con n numero di vertici e m numero di archi. Questo tipo di visita e' utile per capire quale sia il cammino piu' breve tra due nodi nel caso ci sia una BFS partita dal nodo di inizio: si crea l'albero e poi, partendo dal nodo finale, tramite una funzione ricorsiva che a ritroso arriva fino al nodo di inizio si può stampare il cammino più corto.

9.5.2 Visita in profondità (DFS)

Visita i nodi andando il più possibile lontano dalla sorgente e a differenza del BFS arriva a tutti i nodi del grafo, non solo quelli collegati ad un nodo. Per questo motivo la funzione crea un foresta ovvero un insieme di alberi. L'algoritmo e' ricorsivo e per ogni nodo riempie due paramentr: un *discorvery time* ovvero il momento in cui e' stato scoperto e un *finish time* ovvero quando termina la visita del nodo. La visita funziona che parte dal nodo iniziale al momento 1 poi passa ad un altro nodo fino a quando no si arriva ad un nodo che non ha piu' nodi non ancora scoperti disponibili. A quel punto il tale nodo viene chiuso l'esecuzione torna sul nodo precedente e se ha altri nodi adiacenti inesplorati salta a loro se no anche esso viene chiuso e così via fino a

quando non tutti i nodi sono chiusi. Gli alberi vengono creati come prima ma essendo che si guardano tutti i nodi in casi di nodi scollegati o cappi vengono restituiti piu' alberi. Per quanto riguarda la complessità essa rimane invariata rispetto al BFS perche' i passaggi di inizializzazione dei campi necessari allo svolgimento sono quelli piu' dispendiosi. Grazie ai campi **discorvery time** e **finish time** e le relazione tra quelli di due nodi si possono avere diverse informazioni:

- $[u.dt,\ u.ft]$ e $[u.dt,\ u.ft]$ sono **disgiunti** allora u e v non sono discendenti.
- $[u.dt,\ u.ft]$ e' contenuto in $[u.dt,\ u.ft]$ allora u e' discendente di v.
- $[u.dt,\ u.ft]$ contiene $[u.dt,\ u.ft]$ allora v e' discendente di u.
- $v.dt < u.dt \land u.ft < v.ft$ allora l'arco (u, v) e' all'**indietro**.
- $u.dt < v.dt \land v.ft < u.ft$ allora l'arco (u, v) e' in **avanti**.
- $v.ft < u.dt$ allora l'arco (u, v) e' **di attraversamento a sinistra**.

Per quanto riguarda le applicazioni questa visita rende particolarmente facile **Verificare DAG (Direct Acyclic Graph)** ovvero verificare che non ci siano archi all'indietro e fare l'**ordinamento topologico** nella sottocategoria dei DAG. L'ordinamento topologico è interessante perchè può essere interpretato come dipendenze logiche tra eventi tipo catena di montaggio. L'algoritmo si basa appunto su un iniziale DFS e ritorno una lista che viene fatta mettendo in ordine decrescente i finish time di ogni nodo, in modo pratico si fa una head-push ogni volta che un nodo viene chiuso.

9.6 Componenti connesse (grafi non orientati)

Due vertici u e v di un grafo sono **connessi** quando e' possibile raggiungere v partendo da u, questa definizione induce un possibile partizionamento del grafo in insiemi di nodi connessi tra di loro. Per risolvere questo problema basta etichettare inizialmente tutti i nodi con un valore di "non visitato" poi per ogni nodo non ancora visitato si chiama una funzione ricorsiva che etichetta con un valore (che per ogni insieme differente deve cambiare) il nodo poi riempi il campo *parent* dei nodi con il nodo iniziale e la chiama ricorsivamente sugli stessi.

9.7 Componenti fortemente connesse (grafi orientati)

Si esegue un iniziale partizionamento del grafo di tutti i vertici fortemente connessi dal dato elemento, per fare questo basta calcolare $D(x)$ ovvero tutti i discendenti del nodo x e $A(x)$ ovvero l'insieme di tutti i nodi da cui si può raggiungere x. Per calcolare $D(x)$ basta fare una visita qualunque per calcolare $A(x)$ basta invertire l'orientamento delle freccie e fare una nuova ricerca. A questo punto per avere il partizionamento cercato è $D(x) \cap A(x)$. Per risolvere il problema di partizionare l'intero grafo tra i suoi elementi fortementi connessi tra di loro basta iterare l'algoritmo per ogni nodo e trovare l'intersezione tra tutti i nodi. La complessità computazionale è di $O(nm + n^2)$ per cui in realtà esistono algoritmi più efficienti.

9.8 Minimum Spanning Tree

Il problema è quello di voler determinare come connettere diversi elementi tra di loro usando meno collegamenti possibili all'interno di un grafo non orientato, connesso e pesato. E' come se si volesse collegare con la fibra ottica un paese usando meno cavo possibile. Per trovare la combinazione giusta bisogna trovare l'albero di copertura minore possibile, per **albero di copertura** si intende un sottografo di un grado non orientato e connesso che contiene tutti i nodi del grafo, anch'esso è pesato ma come collegamenti ha un sotto insieme di quelli di partenza. Per trovare l'**albero di copertura di peso minimo** bisgna trovare l'albero che minimizza il peso complessivo. Precisazione importante è quella che il problema non sempre ha una soluzione univoca e il numero necessario per collegare n nodi è necessario avere $n - 1$ cammini. Inoltre, un arco è definito **sicuro** se aggiungendolo ad un MST l'unione rimane un MST quindi nella soluzione si devono prediligere questi archi. Il **metodo generico** per risolvere il problema è quello di creare un albero e unirlo sempre con archi sicuri.

9.8.1 Regole del ciclo e del taglio

Un **taglio** di un grafo non orientato è una partizione del grafo in due sottinsiemi disgiunti e un **arco leggero** è l'arco che che costa meno e che collega i due insiemi disgiunti dal taglio; un arco **attraversa il taglio** se dato l'arco u, v allora $u \in S \land v \in V - S$ con V il grafo iniziale e $V - S$ il sottoinsieme post taglio. La **regola del taglio** vuole che scelto un taglio nel grafo che non "spezza" archi già considerati come ottimi e si può prendere come ottimo quello di costo minore. La **regola del ciclo** vuole che dato un ciclo senza archi già individuati come pessimi possiamo considerare pessimo l'arco con peso peggiore. Applicando queste regole si puoò individuare il MST però bisogna capire come e quando applicare queste tecniche.

9.8.2 Algoritmo di Kruskal

Analizza gli archi in costo crescente e utilizza un approccio greedy che si basa sulle union find: se i due nodi del collegamento appartengono gia' allo stesso insieme allora esso e' un arco da scartare (si tratta di un ciclo) mentre se i due nodi sono disgiunti allora e' uin ponte tra le due porzioni di taglio per cui e' il percorso ottimale. In tutto questo per determinare l'MST ogni volta che si fa una union tra due insieme anche i nodi vengono uniti nell'albero finale Il costo totale è $\Theta(m\ log(n))$.

9.8.3 Algoritmo di Prim

Questo algoritmo parte da un nodo e taglia i suoi cammini, procede sempre e solo con la regola del taglio e si 'espande' sempre il taglio tenendo da una parte i nodi già nell'albero (ovvero quelli esplorati) e dall'altra del taglio quelli ancora da inserire. Per implementare questo algoritmo si usano le code con priorità: partendo dal primo nodo si inseriscono nella coda coppie $(nodo\ di\ arrivo, distanza)$ possibili (ovvero i collegamenti dal nodo in analisi) e iterativamente questo processo viene compiuto per ogni nodo secondo l'ordine nella lista con priorita'. A questo punto si capisce perche' nell'implementazione della coda con priorita' e' necessario prevedere il caso in cui un elemento gia' inserito sia reinserito ma con un peso differente: nel caso si sia trovata una strada piu' efficiente per un nodo esso dovra' cambiare di posizione nella coda. La complessità di questo algoritmo è di $\Theta(m\ log(n))$.

9.9 Camminimi minimi

Considerando un grafo orientato si vuole trovare il cammino di costo minimo che collega due nodi dati in input. Ci sono diverse formulazioni del problema oltre a quella appena fornita, per esempio il *single surce shortest path* che dato un solo punto deve restituire tutti i cammini minimi da esso agli altri oppure l'*All-pairs shortest path* ovvero determinare il minimo cammino tra tutte le coppie di punti. Ci sono dei casi in cui non si può trovare un cammino minimo, principalmente solo due: quando i due punti non sono raggiungibili (tipo alcuni nodi di grafi orientati) oppure cammini con costo negativo all'interno di un ciclo che complessivamente ha un costo negativo perchè basta continuare a girare nel ciclo abbassando il costo. Ci sono alcune cose tecniche da sapere a riguardo di questo problema, per esempio se due nodi sono collegati allora esiste sempre un cammino semplice (senza cicli) si costo minimo. Un'altra proprietà del cammino ottimale è quella di essere anche il cammino ottimale per collegare il punto di inizio con qualsiasi altro nodo intermedio a quello finale. Inoltre si possono costruire **Alberi di costo minimo** che contengono il cammino minimo dal nodo di partenza a qualsiasi altro nodo. Ci sono anche alcune utili **proprietà della distanza tra vertici** tra cui:

- $d_{vv} = 0\ \forall v$
- $d_{xy} = min_{(z,y)\in E}(d_{xz} + w(z,y))$ ovvero se esite un cammino da x a y allora esiste almeno un vertice z che è l'ultimo arco del camminimo minimo prima di y. Questa proprietà può suggerire un approccio divide et impera tramite una chiamata ricorsiva ma questo aprroccio in realtà non funziona.
- $d_{xy} = \infty$

9.9.1 Algoritmo di Belman Ford

Questo algoritmo risolve il problema nella forma *single surce shortest path*: data la sorgente s mantiene una stima per eccesso $D_{sv} \geq d_{sv}$ del cammino minimo ad ogni nodo v, al termine dell'esecuzione del problema le stime saranno precise per cui $D_{sv} = d_{sv}$ sse non ci sono cicli di costo negativo. Inizialmente pone $D_{ss} = 0$, $d_{sv} = \infty\ \forall s\neg = v$, successivamente esegue dei *rilassamenti* che migliorano le approssimazioni. Questi rilassamenti sono piuttosto intuitivi: se c'è un cammino tra un nodo intermedio tra s e v chiamato D_{su} che sommato al cammino tra u e v risulta minore di D_{sv} allora si sostituisce il valore della somma essendo un cammino minimo. Per eseguire un ciclo di rilassamento bisgna anallizzare tutti gli archi del grafo vedendo se permettono un effettivo rilassamento. A questo punto tocca capire quante iterazioni di rilasamenti devono essere fatti per arrivare ad una convergenza sicura, la risposta è che dopo k iterazioni si è trovato il cammino minimo che attraversa al massimo k archi. Per cui essendoci al massimo $n-1$ archi in un grafo allora alla $n-1$ ierazione avremo coperto tutti i cammini possibili e trovato dunque il minimo, il costo computazionale è $O(nm)$. In realtà una possibile modifica che lo rende più efficiente è terminare non sempre a $n-1$ ierazioni ma quando non si effettuano più rilassamenti, un'altra modifica è controllare se ci sono cicli negativi: basta, una volta finito il rilassamento, controllare se è possibile eseguirne un altro e in caso affermativo c'è un ciclo di costo negativo.

9.9.2 Algoritmo di Dijkstra

Anche questo algoritmo è necesario a risolvere il *single surce shortes path* che però è più efficiente si quello di Belman-Ford ma non funziona in caso di cammini negativi. Il lemma su cui si basa questo algoritmo è che dato il grafo senza cammini negativi e un albero di cammini minimi dalla radice alle foglie allora il cammino tra una foglia e un nodo non appartenete all'albero è proprio quello dalla foglia al nodo. L'algoritmo viene implementato tramite una coda con priorita' che resituisce sempre il nodo con distanza minima disponibile e li

analiza tutti i collegamenti ad essi andando am mettere nella coda con priorita' i nodi man mano che vengono scoperti.

9.9.3 Algoritmo di Floyd e Warshall

Questo algoritmo risolve il problema dell'*all-pairs shortest paths* e inizialmente e' necessario capire come potre rappresentare la soluzione ovvero voler rappresentare sia le distanze che i cammini tutte le copiie di nodi. Per rappresentare la distanza tra coppie di punti la matrice e' la struttura dati miliore mentre per rappresentare ogni cammino il metodo migliore e' anche in questo caso sempre una matrice che per ogni coppia di partenza-destinazione contiene il primo passo del cammino migliore per cui si andra' iterativamente a cambiare cella fino ad arrivare al nodo finale. Gia' si puo' intuire che questo algoritmo si basa molto sulla programmazione dinamica e iterativamente calcola D^k_{xy} ovvero il cammino tra x e y usando k nodi con $k = 1, ..., n$ e alla fine la soluzione sara' proprio D^n_{xy}. L'inizializzazione e' semplice perche' D^0_{xy} e' 0 se $x = y$, $w(x, y)$ se sono collegati mentre ∞ se non sono collegati; a questo punto tocca capire come calcolare tutti gli altri problemi dando pero' per scontato che il problema $k - 1$ sia stato risolto: la soluzione e' $D^k_{xy} = minD^k_{xy} - 1, D^k_{xk} - 1 + D^k_{ky} - 1$ e a questo punto si puo' costruire l'algoritmo. Si inizializza una matrice 3 dimensionale che ha lati: n, n, n+1 le righe sono la partenza le colonne l'arrivo e la profondita' e' la pesantezza del cammino da $x \to y$; la prima facciata e' D^0_{xy} con x,y che sono i due possibili vertici (cosi' si analizzano tutte le combinazioni). A questo punto con un ciclo diverso si itera la formula precedente per ogni $n + 1$ faccia del cubo e su ogni cella della singola faccia. L'ultima facciata pero' non ci offre anche un modo di trovare un cammino per cui e' necessatrio utilizzare una matrice d'appoggio che tutte le volte che si trova un cammino migliore nel cubo nella matrice d'appoggio si segna il nodo successivo a quello analizzato. Dunque la complessita' computazionale e' $O(n^3)$, si puo' effettuare una piccola ottimizzazione a livello memonico infatti al posto di usare un cubo si puo' usare una tabella normale che cambia ad ogni iterazione ma il valore k e' ancora necessaria per la quantita' di iterazioni. Inoltre si possono anche individuare i cicli negativi nel grafo: a iterazione finita basta controllare se la distanza tra un vertice e se stesso e' inferiore a 0.